

# Computing with Python

CS0079

Module 3 Submodule 1



# Subtopic 1

# Python Data Structures

After this submodule, the learner should be able to:

- Construct strings to access and perform string operations
- Construct dictionaries to access entries and use its operations
- Construct lists to access and manipulate items in the lists
- Construct tuples and use its operations



FEU ALABANG



FEU DILIMAN



FEU TECH

*Technology Driven by Innovation*

# Strings

- represent plain text.

Recall: Strings should be enclosed in either single or double quotations.

The value 2 is different from "2"

```
In [1]: 1 type('I')
```

```
Out[1]: str
```

```
In [2]: 1 type(" ")
```

```
Out[2]: str
```

```
In [3]: 1 type("2")
```

```
Out[3]: str
```

```
In [4]: 1 type(2)
```

```
Out[4]: int
```

```
In [5]: 1 2 - 2
```

```
Out[5]: 0
```

```
In [6]: 1 "2" - 2
```

```
-----  
TypeError
```

```
<ipython-input-6
```

```
----> 1 "2" - 2
```



# Strings: Finding Length

`len()`

- get the number of characters in the string.

```
In [7]: 1 len("Hey there!")  
Out[7]: 10
```

The value 10 indicates that there are 10 characters in the string "Hey there!"

Note that this includes the space " " and the exclamation mark "!"



FEU ALABANG



FEU DILIMAN



FEU TECH

*Technology Driven by Innovation*

# Strings: Index

- indicates the location of the individual characters in a string.

| Character | H | e | y |   | t | h | e | r | e | ! |
|-----------|---|---|---|---|---|---|---|---|---|---|
| Index     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

```
In [8]: 1 a_string = "Hey there!"  
        2 a_string[0]           # access the character at index 0  
  
Out[8]: 'H'
```



# Strings: Index

- indicates the location of the individual characters in a string.

| Character | H | e | y |   | t | h | e | r | e | ! |
|-----------|---|---|---|---|---|---|---|---|---|---|
| Index     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

```
In [8]: 1 a_string = "Hey there!"
        2 a_string[0]
Out[8]: 'H'
```

| Character | H   | e  | y  |    | t  | h  | e  | r  | e  | !  |
|-----------|-----|----|----|----|----|----|----|----|----|----|
| Index     | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

```
In [11]: 1 a_string = "Hey there!"
         2 a_string[-1]
Out[11]: '!'
```

Type in the name of the string and then its index enclosed in square brackets to access a character given its index.



# Strings: Slicing

| Character | H | e | y |   | t | h | e | r | e | ! |
|-----------|---|---|---|---|---|---|---|---|---|---|
| Index     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

- Accessing a part of the string through index enclosed in square brackets.

```
In [12]: 1 a_string[4:9]  # access the character starting from index 4 until index 8
Out[12]: 'there'
```

Notice that the end index specified is until 9 but the output only returned until index 8.

This means that the range specified in square bracket excludes the upper bound.



# Strings: Slicing

| Character | H | e | y |   | t | h | e | r | e | ! |
|-----------|---|---|---|---|---|---|---|---|---|---|
| Index     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

The syntax for slicing in Python is [start:end:step]

```
In [13]: 1 a_string[4:9:2]      # access the character starting from index 4 until index 8 in steps of 2
Out[13]: 'tee'

In [14]: 1 a_string[4:]        # access the character starting from index 4 until end of the string
Out[14]: 'there!'

In [15]: 1 a_string[:4]        # access the character starting from beginning until index 3
Out[15]: 'Hey '

In [16]: 1 a_string[::2]       # access the character starting from beginning until end in steps of 2
Out[16]: 'Hytee'

In [17]: 1 a_string[4:-1]      # access the character starting from index 4 until the last character.
Out[17]: 'there'

In [18]: 1 a_string[::-1]      # output the string in reverse order
Out[18]: '!ereht yeH'
```



# Strings: Concatenation

This is indicated by the symbol + performed on strings.

```
In [1]: a_string = "Budget: "  
        another_string = "4000"  
        print(a_string + " " + another_string)
```

Budget: 4000

As indicated in the **TypeError** output, it cannot concatenate a string with an integer.

```
In [2]: a_string = "Budget: "  
        another_string = 4000  
        print(a_string + " " + another_string)
```

-----  
**TypeError**

Traceback (mo

Input In [2], in <cell line: 3>()

1 a\_string = "Budget: "

2 another\_string = 4000

----> 3 print(a\_string + " " + another\_string)

**TypeError:** can only concatenate str (not "int") to str



FEU ALABANG



FEU DILIMAN



FEU TECH

# Strings: Concatenation

Using a built-in function **str()** to convert an integer value.

```
In [3]: print("Budget: " + str(4000))  
Budget: 4000
```

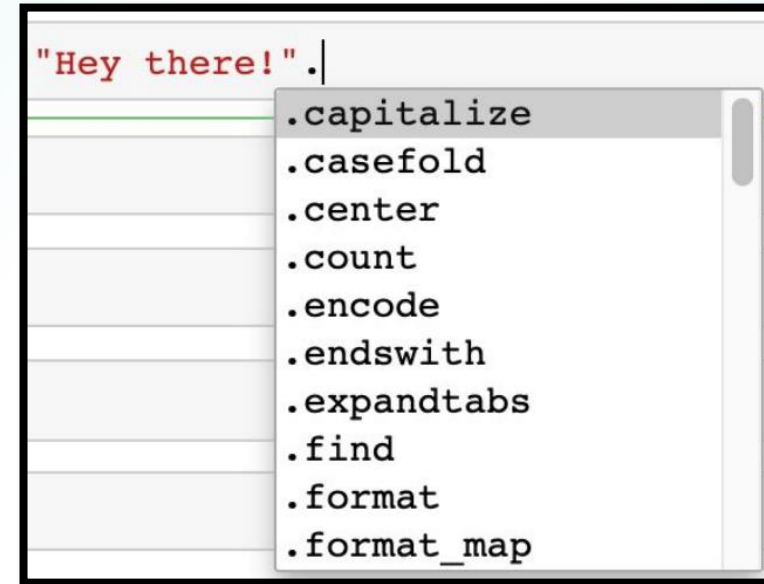
Determine the data type using **type()** on **4000**.

```
In [4]: type(str(4000))  
Out[4]: str
```



# Strings: Methods

In Python, a string is an object that has various methods that can be used to manipulate it.



To have an idea of how a certain string method works, use the command **help()**.

```
1 help("Hey there!".capitalize)
```

Help on built-in function capitalize:

capitalize() method of builtins.str instance  
Return a capitalized version of the string.

# Strings: Methods

Some example use of string methods:

```
In [1]: msg = "FEU Institute of Technology"  
        msg.upper()
```

```
Out[1]: 'FEU INSTITUTE OF TECHNOLOGY'
```

```
In [2]: msg.count("e")
```

```
Out[2]: 2
```

```
In [3]: msg.replace(" institute of technology", "TECH")
```

```
Out[3]: 'FEU Institute of Technology'
```

```
In [4]: msg
```

```
Out[4]: 'FEU Institute of Technology'
```

**Note:** Using string methods does not change the original string.



FEU ALABANG



FEU DILIMAN



FEU TECH

*Technology Driven by Innovation*

# Lists

- is a sequence of values.
- the values in a list are called its elements or items.
- is defined by enclosing its items in square brackets, separated by commas.
- a list can hold any type of data

```
In [1]: 1 numbers = [10,20,30,40,50]
        2 states = ["solid", "liquid", "gas"]
        3 list_boolean = [True, False]

In [2]: 1 mixed_type_list = [1, "way", 2, "say", 3, "words"]

In [3]: 1 list_of_lists = [numbers, states]
        2 list_of_lists

Out[3]: [[10, 20, 30, 40, 50], ['solid', 'liquid', 'gas']]
```



# Lists: Operations

- use of `len()`
- use of (negative) index
- slicing
- concatenation (+)

```
states = ["solid", "liquid", "gas"]
```

```
mixed_type_list = [1, "way", 2, "say", 3, "words"]
```

```
In [4]: 1 states[1]
```

```
Out[4]: 'liquid'
```

```
In [5]: 1 states[-1]
```

```
Out[5]: 'gas'
```

```
In [6]: 1 states[0]
```

```
Out[6]: 'solid'
```

```
In [7]: 1 states[:2]
```

```
Out[7]: ['solid', 'liquid']
```

```
In [8]: 1 len(states)
```

```
Out[8]: 3
```

```
In [10]: 1 mixed_type_list + states
```

```
Out[10]: [1, 'way', 2, 'say', 3, 'words', 'solid', 'liquid', 'gas']
```



# Lists: Check if an item is present

- `in` - to whether an exists in a list.

```
states = ["solid", "liquid", "gas"]
```

```
In [11]: 1 "solid" in states
```

```
Out[11]: True
```

```
In [12]: 1 "plasma" in states
```

```
Out[12]: False
```



# Lists: Methods

- **append()** - add an item at the end of list.
- **insert()** - insert an item at a given index.
- **remove()** - remove first occurrence of a string.

```
In [1]: exam_scores = [90,83,85,77]
        exam_scores.append(91)
        exam_scores
```

```
Out[1]: [90, 83, 85, 77, 91]
```

```
In [2]: pokemon = ["Pikachu", "Oshawott", "Goodra"]
        pokemon.insert(1,"Oshawott")
        pokemon
```

```
Out[2]: ['Pikachu', 'Oshawott', 'Oshawott', 'Goodra']
```

```
In [3]: pokemon.remove("Oshawott")
        pokemon
```

```
Out[3]: ['Pikachu', 'Oshawott', 'Goodra']
```

Removes the first occurrence of Oshawott.



# Lists: String to List and Vice Versa

```
In [4]: list("Watermelon") # convert a string into a list of characters
```

```
Out[4]: ['W', 'a', 't', 'e', 'r', 'm', 'e', 'l', 'o', 'n']
```

```
In [5]: FEU_School_list = ["Alabang", "Diliman", "Tech"]  
        " ".join(FEU_School_list) #join the string elements of list in one string
```

```
Out[5]: 'Alabang Diliman Tech'
```

```
In [6]: "_".join(FEU_School_list) # join the string elements of a list in one string
```

```
Out[6]: 'Alabang_Diliman_Tech'
```

```
In [7]: FEU_string = "_".join(FEU_School_list)  
        FEU_string.split("_") # split the characters of a string using an underscore.
```

```
Out[7]: ['Alabang', 'Diliman', 'Tech']
```



# Tuple

- is a sequence of values.
- Using a pair of parentheses `()`, elements are separated by commas.
- Operations like getting the length, indexing, slicing, using `+` and `in` are the same as those in strings and lists.

```
a_tuple=(1,1000,2,1000,3,1000)
```

```
1 len(a_tuple)
```

```
6
```

```
1 a_tuple[2:]
```

```
(2, 1000, 3, 1000)
```

```
1 a_tuple.count(1000)
```

```
3
```



# Tuple: Tuple vs List

- Lists are mutable, tuples are immutable
- Mutable - items can be changed **in-place**

```
a_tuple=(1,1000,2,1000,3,1000)
```

```
In [12]: 1 a_list = [1,5,1,6,1,7]
          2 a_list[0] = 2
          3 a_list
Out[12]: [2, 5, 1, 6, 1, 7]
```

```
In [13]: 1 a_tuple[0]=2
-----
TypeError
<ipython-input-13-e9c
----> 1 a_tuple[0]=2
```



# Tuple: Tuple vs List

- Lists are usually homogeneous, tuples are usually heterogeneous.
- Homogeneous - collection with same type
- Heterogeneous - collection with different types

```
1 # illustration of homogeneous: a list of school items
2 ["paper", "pencil", "highlighter", "erase"]
```

```
['paper', 'pencil', 'highlighter', 'erase']
```

```
1
2 # a list of (thing, number) pairs
3 [("paper", 10), ("pencil", 20), ("highlighter", 5), ("erase", 8)]
```

```
[('paper', 10), ('pencil', 20), ('highlighter', 5), ('erase', 8)]
```



# Dictionary

- A dictionary is a general version of a list.
- In a list, the indices have to be integers; in a dictionary, they can be (almost) any type.
- Dictionaries are indexed by **keys**.
- A dictionary comprises of key-value pairs, and each key maps to a comma-separated **key:value** pairs.



FEU ALABANG



FEU DILIMAN



FEU TECH

*Technology Driven by Innovation*

# Dictionary: Indexing by keys

- Dictionaries are declared by (comma-separated) key:value pairs enclosed in square brackets.

```
In [4]: 1 sample_dict = {"one": "uno", "two": "dos", "three": "tres"}
        2 sample_dict["one"]
Out[4]: 'uno'

In [5]: 1 sample_dict["two"]
Out[5]: 'dos'

In [7]: 1 sample_dict["four"]

-----
KeyError                                Traceback (most recent call last)
<ipython-input-7-4844af9b4609> in <module>()
----> 1 sample_dict["four"]

KeyError: 'four'
```



# Dictionary: get all keys, get all values

- **keys()** method to get all the keys in a dictionary.
- **values()** method to get all the values in a dictionary.
- **items()** method to get all the key-value pairs in a dictionary.

```
sample_dict = {"one": "uno", "two": "dos", "three": "tres"}
```

```
In [4]: 1 sample_dict.keys()
```

```
Out[4]: dict_keys(['one', 'two', 'three'])
```

```
In [5]: 1 sample_dict.values()
```

```
Out[5]: dict_values(['uno', 'dos', 'tres'])
```

```
In [6]: 1 sample_dict.items()
```

```
Out[6]: dict_items([('one', 'uno'), ('two', 'dos'), ('three', 'tres')])
```



# Dictionary: adding elements

- Sample code to define an empty dictionary and add an element.

```
In [7]: 1 delicacies = {}                                # an empty dictionary
        2 delicacies["puto"] = "Calasiao"
        3 delicacies["bibingka"] = "Vigan"
        4 delicacies["ensaymada"] = "Bulacan"
        5 delicacies

Out[7]: {'puto': 'Calasiao', 'bibingka': 'Vigan', 'ensaymada': 'Bulacan'}
```



# Dictionary: from list of pairs to dict

- Turning a list of tuples with (**key,value**) pair to a dictionary.

```
In [8]: 1 a_list=[("puto","Calasiao"), ("bibingka","Vigan"), ("ensaymada","Bulacan")]  
        2 a_list
```

```
Out[8]: [('puto', 'Calasiao'), ('bibingka', 'Vigan'), ('ensaymada', 'Bulacan')]
```

```
In [9]: 1 dict(a_list)
```

```
Out[9]: {'puto': 'Calasiao', 'bibingka': 'Vigan', 'ensaymada': 'Bulacan'}
```



# Dictionary: checking existence of a key or value

- The `in` operator when used on dictionaries search occurrence from the keys.

```
{'puto': 'Calasiao', 'bibingka': 'Vigan', 'ensaymada': 'Bulacan'}
```

```
In [10]: 1 "puto" in delicacies
```

```
Out[10]: True
```

```
In [11]: 1 "pichi-pichi" in delicacies
```

```
Out[11]: False
```



# Dictionary: store dictionary keys in a list

- We can use the command `list()` to access a list of dictionaries keys.

```
{'puto': 'Calasiao', 'bibingka': 'Vigan', 'ensaymada': 'Bulacan'}
```

```
In [12]: 1 list(delicacies)
```

```
Out[12]: ['puto', 'bibingka', 'ensaymada']
```



# Exercise

- Add "**pichi-pichi**":"Quezon" to our created dictionary, delicacies.

```
1 delicacies
{'puto': 'Calasiao', 'bibingka': 'Vigan', 'ensaymada': 'Bulacan'}
```

```
1 delicacies["pichi-pichi"] = "Quezon"

1 delicacies
{'puto': 'Calasiao',
 'bibingka': 'Vigan',
 'ensaymada': 'Bulacan',
 'pichi-pichi': 'Quezon'}
```



# Q&A SESSION





FEU ALABANG



FEU DILIMAN



FEU TECH

*Technology Driven by Innovation*

# References

1. SPARTA (2023). *SP202: Computing in Python*. Linang: Development Academy of the Philippines. <https://linang.dap.edu.ph>.
2. Scratch, M. (2020). *Python Algorithms: A complete guide to learn python for data analysis, machine learning, and coding from scratch*.
3. Matthes, E. (2019). *Python crash course: A hands-on, project-based introduction to programming*. San Francisco, CA. No Starch Press.
4. Pomperada, J. & Moises, R. (2019). *Fundamentals of python programming*. Manila. MindShaper Co. Inc.
5. Ahmad, I. (2020). *40 Algorithms Every Programmer Should Know: Hone Your Problem-Solving Skills By Learning Different Algorithms And Their Implementation In Python*. Packt Publishing.
6. Codings, Z. (2019). *Computer Programming And Cyber Security For Beginners: This Book Includes: Python Machine Learning, Sql, Linux, Hacking With Kali Linux, Ethical Hacking. Coding And Cybersecurity Fundamentals*.

